

FSD

Exp 2: Perform CRUD operations on files using fs module

1. fs is Node.js's built-in **File System module**. It lets you Create, Read, Update, and Delete (CRUD) files directly from JavaScript code running in Node.
- 2.

What is fs?

- fs stands for **File System**.
- It is a **built-in Node.js core module** — no installation needed (npm install not required).
- It allows JavaScript code running in Node.js to interact directly with files and folders on the computer (something browser JavaScript cannot normally do).
- Almost every backend application needs to read configuration files, write logs, store uploaded files, or manage data — fs is the foundation for all of that.

How to use it:

```
javascript

const fs = require('fs');           // CommonJS
// or
import fs from 'fs';                // ES Modules
```

2. What is CRUD?

Letter	Operation	Meaning
C	Create	Make a new file
R	Read	Open and view file contents
U	Update	Modify existing file content
D	Delete	Remove a file

CRUD is a universal concept — used in databases, APIs, and file systems alike.

3. Two Styles of fs Methods

Explain to students that Node offers **three ways** to do the same operation:

Style	Example Method	Behavior
Callback-based	fs.writeFile()	Asynchronous, non-blocking
Synchronous	fs.writeFileSync()	Blocks code until operation completes

Promise-based	fs.promises.writeFile()	Asynchronous, works with async/await
---------------	-------------------------	--------------------------------------

```
javascript

const fs = require('fs');

fs.writeFile('example.txt', 'Hello World!', (err) => {
  if (err) throw err;
  console.log('File created!');
});
```

4. CREATE — Making a File

Key points to explain:

- writeFile creates the file **if it doesn't exist**.
- If the file **already exists**, it will be **overwritten** completely.
- Always include error handling (err) — file operations can fail (permissions, disk space, etc.)

Sync version:

```
javascript

fs.writeFileSync('example.txt', 'Hello World!');
```

5. READ — Viewing File Contents

```
javascript

fs.readFile('example.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

Key points to explain:

- 'utf8' tells Node to return readable text instead of raw binary data (Buffer).
- Without 'utf8', data will be a Buffer object (show this in class as a demo — good "aha" moment).

Sync version:

```
javascript  
  
const data = fs.readFileSync('example.txt', 'utf8');  
console.log(data);
```

6. UPDATE — Modifying a File

Two approaches — important distinction for students:

a) Overwrite (replace all content)

```
javascript  
  
fs.writeFile('example.txt', 'New content', (err) => {  
  if (err) throw err;  
});
```

b) Append (add to existing content)

```
javascript  
  
fs.appendFile('example.txt', '\nMore content', (err) => {  
  if (err) throw err;  
});
```

7. DELETE — Removing a File

javascript

```
fs.unlink('example.txt', (err) => {  
  if (err) throw err;  
  console.log('File deleted!');  
});
```

Key points to explain:

- Method is called **unlink**, **not** delete (common confusion — delete is a JS keyword, so Node avoids the name clash).
 - Deletion is **permanent** — no recycle bin.
-

8. Modern Style — Promises + Async/Await

javascript



```
const fs = require('fs').promises;  
  
async function run() {  
  await fs.writeFile('example.txt', 'Hello!');  
  const data = await fs.readFile('example.txt', 'utf8');  
  console.log(data);  
  await fs.unlink('example.txt');  
}  
  
run();
```

- Cleaner, more readable code than nested callbacks ("callback hell").
- Industry-standard approach in modern Node.js applications.
- Requires try/catch for error handling — good chance to review that concept too.

javascript



```
async function run() {
  try {
    await fs.writeFile('example.txt', 'Hello!');
  } catch (err) {
    console.error('Something went wrong:', err);
  }
}
```

9. Summary Table (Good for Whiteboard/Slide)

Operation	Async Method	Sync Method	Promise Method (fs.promises)
Create	fs.writeFile()	fs.writeFileSync()	fs.writeFile() (awaited)
Read	fs.readFile()	fs.readFileSync()	fs.readFile() (awaited)
Update	fs.appendFile()	fs.appendFileSync()	fs.appendFile() (awaited)
Delete	fs.unlink()	fs.unlinkSync()	fs.unlink() (awaited)

Note:

CREATE block

javascript

```
fs.writeFile('example.txt', 'Hello World!', (err) => {
```

- Calls `writeFile` to create a new file named `example.txt`.
- Writes the string `'Hello World!'` into it.
- The third argument `(err) => {...}` is a **callback function** — code that runs *after* the write finishes.
- `err` will be `null` if it succeeded, or an error object if something went wrong.

javascript

```
if (err) throw err;
```

- If there was an error (e.g., no write permission ↓), stop execution and throw it.

javascript

```
console.log('File created!');
```

- Confirms success by printing to the console — this only runs once the file is actually written, because it's inside the callback.

Cell — Update only:

python

```
%%writefile update.js
const fs = require('fs');

// UPDATE (overwrite)
fs.writeFile('example.txt', 'This is the updated content.', (err) => {
  if (err) throw err;
  console.log('File overwritten (updated)!');
});
```

Cell — Update by appending instead:

python

```
%%writefile update.js
const fs = require('fs');

// UPDATE (append)
fs.appendFile('example.txt', '\nThis line was added.', (err) => {
  if (err) throw err;
  console.log('File updated (appended)!');
});
```

javascript

```
fs.readFile('example.txt', 'utf8', (err, data) => {
```

- Now reads the same file back.
- `'utf8'` tells Node to return the content as **readable text** instead of raw binary (`Buffer`).
- This callback receives **two things**: `err` (any error) and `data` (the file's actual content).

javascript

```
    console.log('File content:', data);  
  });  
});
```

- Prints the file's content to the console.
- The two closing `});` match the two functions we opened — first closes `readFile`, second closes `writeFile`.



Cell — Delete:

python

```
%writefile delete.js  
const fs = require('fs');  
  
// DELETE  
fs.unlink('example.txt', (err) => {  
  if (err) throw err;  
  console.log('File deleted!');  
});
```

Note:

Microtasks (run first, higher priority)

#	Source	Example	Notes
1	process.nextTick	process.nextTick(fn)	Node-specific, runs before all other microtasks
2	Promise callbacks	Promise.then / .catch / .finally	Standard JS microtask queue
3	async/await continuation	code after await inside async fn	Built on top of Promises internally
4	queueMicrotask()	queueMicrotask(fn)	Direct way to schedule a microtask manually

Macrotasks (run after all microtasks are empty)

#	Source	Example	Event loop phase
1	setTimeout	setTimeout(fn, ms)	Timers phase
2	setInterval	setInterval(fn, ms)	Timers phase
3	File / network I/O	fs.readFile, http requests	Poll phase
4	setImmediate	setImmediate(fn)	Check phase (Node-specific)
5	I/O event callbacks	socket close, UI events (browser)	Close callbacks phase

The Program (recap)

```
javascript
console.log('1: Start (sync)');

setTimeout(() => {
  console.log('2: Inside setTimeout (macrotask - runs LAST)');
}, 0);

Promise.resolve().then(() => {
  console.log('3: Inside Promise.then (microtask - runs BEFORE setTimeout)');
});

fs.readFile(__filename, () => {
  console.log('4: Inside fs.readFile callback (I/O - runs with macrotasks)');
});

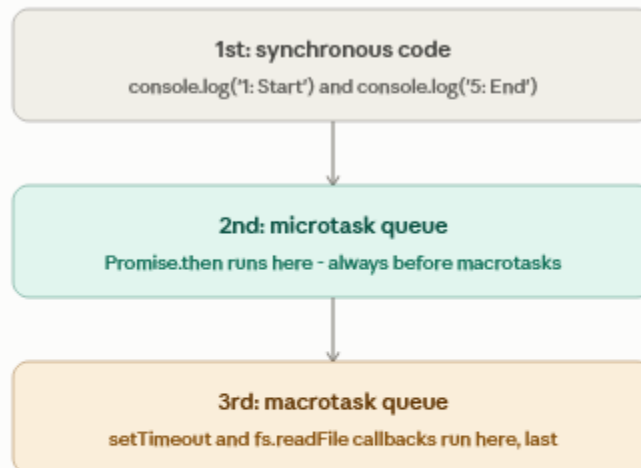
console.log('5: End (sync)');
```

```

1: Start (sync)
5: End (sync)
3: Inside Promise.then (microtask - runs BEFORE setTimeout)
4: Inside fs.readFile callback (I/O - runs with macrotasks)
2: Inside setTimeout (macrotask - runs LAST)

```

Priority	Type	What it includes	Example	Runs when
1st	Synchronous code	Plain statements, normal <code>console.log()</code>	<code>console.log('Start')</code>	Immediately, top to bottom — nothing can interrupt it
2nd	Microtask	<code>Promise.then()</code> , <code>async/await</code> continuations, <code>process.nextTick()</code>	<code>Promise.resolve().then(...)</code>	Right after all sync code finishes, before any macrotask
3rd	Macrotask	<code>setTimeout()</code> , <code>setInterval()</code> , file/network I/O (<code>fs.readFile</code>)	<code>setTimeout(fn, 0)</code>	Only after the microtask queue is completely empty



Output order for the numbered example:

1 → 5 → 3 → 4 → 2

(sync lines, then Promise.then, then fs.readFile, then setTimeout)

Rule: sync always first, microtasks always drain before macrotasks

EXP1.3 :Visualize the event loop using setTimeout, setImmediate, and process.nextTick

```
javascript

console.log('1: Start (sync)');

setTimeout(() => {
  console.log('2: setTimeout (macrotask - timers phase)');
}, 0);

setImmediate(() => {
  console.log('3: setImmediate (macrotask - check phase)');
});

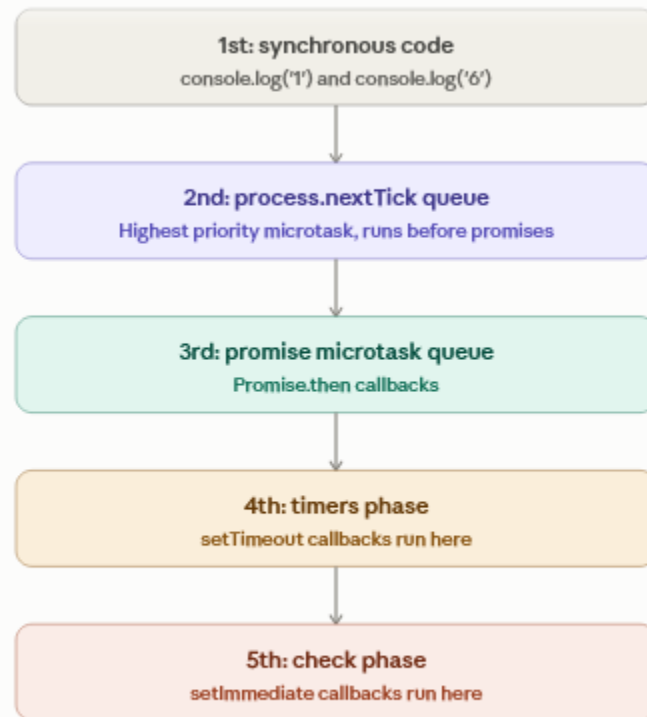
process.nextTick(() => {
  console.log('4: process.nextTick (highest priority microtask)');
});

Promise.resolve().then(() => {
  console.log('5: Promise.then (microtask)');
});

console.log('6: End (sync)');
```

Typical output:

```
1: Start (sync)
6: End (sync)
4: process.nextTick (highest priority microtask)
5: Promise.then (microtask)
2: setTimeout (macrotask - timers phase)
3: setImmediate (macrotask - check phase)
```



Output order for the code above:

1 → 6 → 4 → 5 → 2 → 3

Priority	Type	Example	Runs when	Output
1st	Synchronous code	<code>console.log(...)</code>	Immediately, top to bottom	1, 6
2nd	<code>process.nextTick</code> queue	<code>process.nextTick(fn)</code>	Right after sync code, before promises	4
3rd	Promise microtask queue	<code>Promise.then(fn)</code>	After nextTick queue is empty	5
4th	Timers phase	<code>setTimeout(fn, 0)</code>	After all microtasks are done	2
5th	Check phase	<code>setImmediate(fn)</code>	After timers phase, each loop cycle	3